

AI Architect - Technical Assignment

Duration: 5 days | **Submission:** PDF/Markdown + Architecture diagrams + Code repository

Assignment Overview

You are engaged as the AI Architect for **FinServ Global**, a mid-large financial services company that wants to build an **AI-powered Regulatory Compliance Assistant**. The system must help compliance officers navigate complex regulatory documents (Basel III, MiFID II, RBI guidelines), answer natural-language queries, generate compliance reports, and flag potential violations in real-time transaction data.

Note: Candidates must use only **open-source data, models, and libraries** for the implementation. If a proprietary or paid service is referenced in the architecture design, clearly call it out and provide an open-source alternative that the prototype uses.

Use Case — Expanded Detail

Business Context

FinServ Global operates across **India, EU, and US** markets. Its compliance team of ~40 officers currently spends 60%+ of their time manually searching regulatory PDFs, cross-referencing circulars, and preparing audit-ready reports. Key pain points:

- **Regulatory volume:** 200+ new circulars/amendments per year across Basel III, MiFID II, and RBI master directions
- **Cross-regulation complexity:** A single transaction (e.g., a cross-border derivative trade) can trigger obligations under multiple frameworks simultaneously
- **Audit pressure:** Regulators expect documented evidence of how compliance decisions were reached — traceability is non-negotiable

Target User Personas

Persona	Role	Primary Need
Compliance Officer	Day-to-day checks	“Does this transaction violate any regulation?” — needs fast, cited answers
Compliance Head	Oversight & reporting	Weekly/monthly compliance posture reports, trend analysis

Persona	Role	Primary Need
Internal Auditor	Post-facto review	Audit trail of every AI-assisted decision, with source citations

Core Scenarios the System Must Handle

1. **Natural-language regulatory Q&A** — “What are the capital adequacy requirements for Tier 1 under Basel III as amended in 2023?” → cited, versioned answer
2. **Transaction screening** — Given a transaction payload (amount, counterparty, jurisdiction, instrument type), flag applicable regulations and produce a risk-rated compliance assessment
3. **Regulatory change impact** — When a new RBI circular is ingested, identify which existing compliance policies or transaction types are affected
4. **Report generation** — Generate a structured compliance report for a set of transactions over a period, suitable for submission to an internal audit committee

Sample Transaction Payloads (for 2B)

Candidates should test against scenarios like:

- Cross-border payment of \$2M to a non-KYC verified entity in a high-risk jurisdiction
- Intra-group derivative trade exceeding large exposure threshold
- Retail customer investment in a complex product without appropriateness assessment (MiFID II)
- NBFC lending transaction requiring priority sector reporting (RBI)

Expected Outcome

What a Strong Submission Looks Like

- A **production-aware architecture** — not a tutorial-grade design. The candidate should demonstrate they have thought about what happens at scale, under failure, and over time (regulation changes, model deprecation, cost growth).
- **Working code that reflects the architecture** — the prototype should be a faithful slice of the designed system, not a disconnected Jupyter notebook. Modularity, config-driven behavior, and clear separation of concerns matter.
- **Defensible trade-offs** — every major choice (model, vector DB, framework, cloud service) should come with a “why this over that” rationale grounded in the FinServ context (regulatory sensitivity, latency SLAs, data residency).

What We Do NOT Want

- Generic LangChain tutorials repackaged as architecture
 - Over-reliance on a single LLM provider with no fallback or cost controls
 - Architectures that ignore data sovereignty (e.g., sending RBI circular text to US-hosted APIs without addressing it)
-

Part 1: Solution Architecture & Design (50 points)

1A. End-to-End Architecture Document (25 points)

Produce a detailed architecture document covering:

1. **High-Level Architecture Diagram** showing all major components and their interactions
2. **RAG Pipeline Design**
 - Document ingestion strategy for regulatory PDFs, circulars, and amendments (multi-format, versioned)
 - Chunking strategy with justification (size, overlap, semantic boundaries)
 - Choice of embedding model and vector database – justify with trade-off analysis
 - Retrieval strategy (hybrid search, re-ranking, contextual compression)
 - How you handle document updates and version control in the vector store
3. **LLM Orchestration Layer**
 - Foundation model selection with rationale (cost, latency, accuracy, compliance)
 - Multi-model routing strategy (e.g., smaller model for classification, larger for generation)
 - Prompt engineering framework: system prompts, few-shot templates, chain-of-thought design
 - Guardrails: hallucination detection, output validation, PII redaction, regulatory accuracy checks
4. **Agentic Workflow Design**
 - Design an agent that can: query the knowledge base, cross-reference multiple regulations, pull transaction data, and generate a compliance assessment
 - Include tool definitions, state management, and error-handling strategy
 - Specify the agentic framework you'd use and why

1B. Infrastructure & Non-Functional Requirements (15 points)

1. **Cloud Architecture** (pick AWS, Azure, or GCP):
 - Deployment topology with Kubernetes orchestration
 - Auto-scaling strategy for model inference endpoints
 - Cost estimation for 500 concurrent users, 10K queries/day

2. Security & Compliance:

- Data residency and encryption (at rest + in transit)
- Model access controls and audit logging
- How you ensure no regulated data leaks to external LLM providers

3. Observability:

- Monitoring stack for LLM performance (latency, token usage, cost)
- Evaluation pipeline: automated quality scoring of LLM responses
- Drift detection for retrieval relevance over time

1C. Architecture Decision Records (10 points)

Write **3 ADRs** for key decisions in your design. Each ADR must include:

- Context, Decision, Alternatives Considered, Consequences, and Status

Suggested topics (pick any 3):

- Vector database selection
- Build vs. buy for the orchestration layer
- Model hosting strategy (self-hosted vs. API)
- Guardrail implementation approach
- Data pipeline orchestration tool choice

Part 2: Hands-On Implementation (50 points)

Build a working prototype that demonstrates core capabilities. Submit a GitHub/GitLab repository.

2A. RAG Pipeline (20 points)

Implement a functional RAG pipeline that:

- Ingests at least 5 sample regulatory documents (you may use publicly available RBI circulars or Basel III excerpts)
- Uses a vector store of your choice
- Supports hybrid search (semantic + keyword)
- Includes a re-ranking step
- Returns cited, source-attributed answers

Evaluation criteria: code quality, modularity, retrieval accuracy, proper error handling.

2B. Agentic Compliance Checker (20 points)

Build an agent (using LangGraph, CrewAI, or framework of your choice) that:

- Accepts a transaction description (e.g., “Cross-border payment of \$2M to a non-KYC entity in a high-risk jurisdiction”)
- Queries the RAG pipeline for relevant regulations
- Produces a structured compliance assessment with: risk rating, applicable regulations, required actions, and citations
- Handles ambiguous or insufficient information gracefully

2C. Evaluation Framework (10 points)

Implement an automated evaluation setup:

- Create a test dataset of 15–20 question-answer pairs with ground truth
- Measure: faithfulness, answer relevance, context precision, context recall (use RAGAS or a custom framework)
- Produce a summary report with scores and failure analysis

Evaluation Rubric

Area	Points	What We're Looking For
Solution Architecture & Design	50	Depth of design, trade-off analysis, practicality
Hands-On Implementation	50	Working code, engineering quality, GenAI best practices
Total	100	

Submission Checklist

- ☐ Architecture document with diagrams (PDF/Markdown)
- ☐ 3 Architecture Decision Records
- ☐ Code repository (GitHub/GitLab) with README, setup instructions, and sample outputs
- ☐ Evaluation report with RAGAS/custom metric scores

This assignment is confidential and intended solely for evaluation purposes. Candidates may use any publicly available tools, libraries, and models. Use of AI coding assistants is permitted but the candidate must be able to explain and defend every design and implementation choice.